

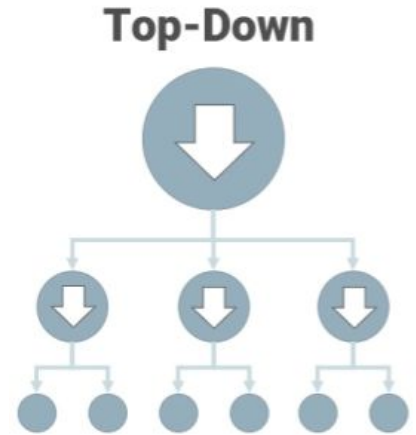
PROGRAMACIÓN DE MICROCONTROLADORES

Modelo top-down

**Carrera de Especialización en Sistemas Embebidos
Facultad de Ingeniería - UBA**

¿Qué es el modelo top-down?

- Es un enfoque de **diseño y desarrollo** de software que se basa en **dividir el proyecto en partes más pequeñas** y manejables.
- Es una estrategia clásica utilizada en ingeniería de software para **construir sistemas complejos**, donde el enfoque principal es la **descomposición jerárquica** del problema **en subproblemas más simples y manejables**.
- La idea central es que el desarrollo **se inicia desde una perspectiva de alto nivel** y luego se profundiza en los detalles.



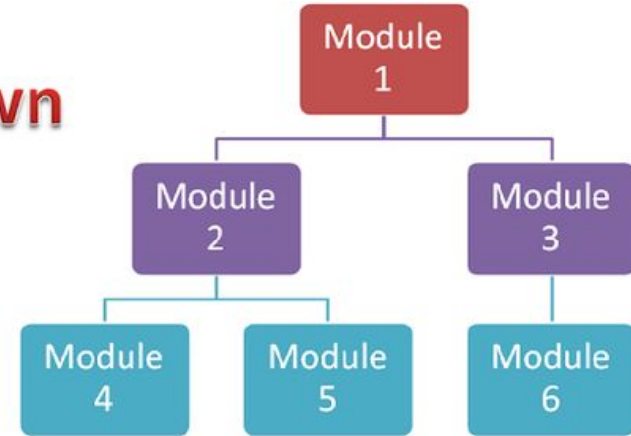
Modelo top-down

- Década de 1970, investigadores de IBM Harlan Mills y Niklaus Wirth.
[Program development by stepwise refinement](#)
- La construcción de programas consiste en una **secuencia de pasos de refinamiento**.
- En cada paso, una tarea dada **se descompone en una serie de subtareas**.
- Cada paso de refinamiento puede ir acompañado de un **refinamiento de la descripción de los datos** que constituyen los medios de comunicación entre las subtareas.
- El refinamiento de la descripción del programa y las estructuras de datos **debe avanzar en paralelo**.

Modelo top-down

- El **grado de modularidad** obtenido determinará la **facilidad o dificultad** con la que un programa puede **adaptarse a cambios o extensiones** del propósito o cambios en el entorno (idioma, computadora) en el que se ejecuta.

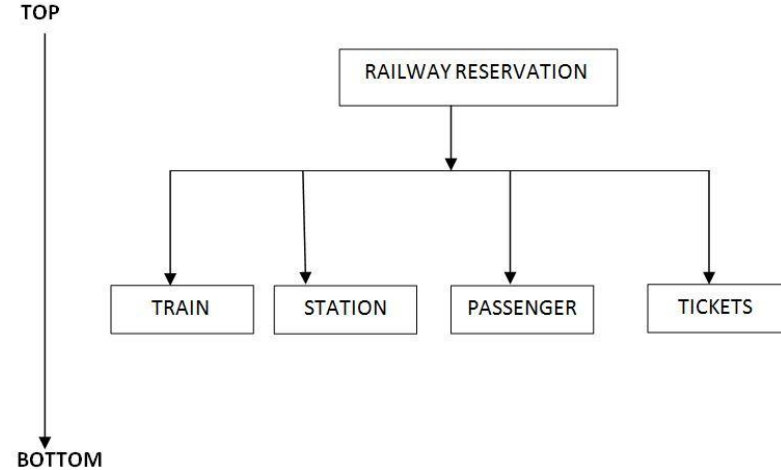
Top Down



- Cada refinamiento implica una serie de **decisiones de diseño** basadas en un conjunto de criterios de diseño. Entre estos criterios se encuentran la **eficiencia, la economía de almacenamiento, la claridad y la regularidad de la estructura**.

Top-down, características

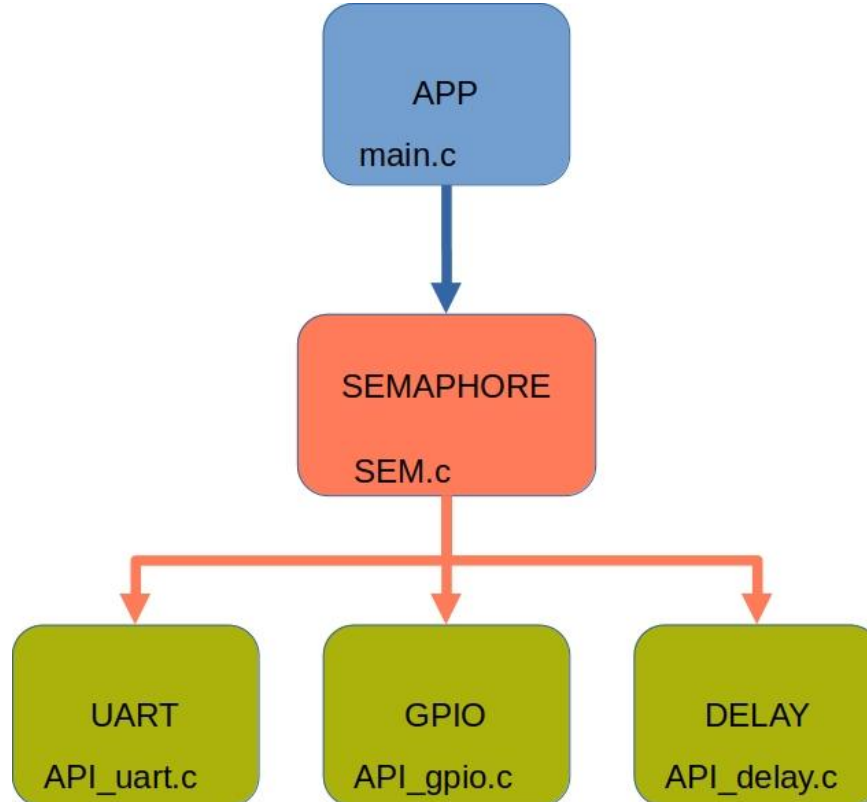
- Formular una **visión general** del sistema sin entrar en detalles específicos.
- **Refinar cada parte** del sistema con más detalles en cada paso o iteración.
- **Continuar definiendo** las partes hasta que toda la especificación sea lo suficientemente detallada para validar el modelo.



Flujo de trabajo típico en el modelo Top-Down

1. **Análisis y diseño de alto nivel:** en esta fase, se identifican los requisitos generales del sistema y se realiza un diseño de alto nivel que describe la estructura y funcionalidad general del software.
2. **Descomposición:** se divide el sistema en módulos o funciones más pequeñas, manteniendo un enfoque en la funcionalidad general del sistema.
3. **Detalle progresivo:** los módulos se siguen dividiendo en partes más pequeñas, lo que lleva a una descomposición jerárquica. Este proceso continúa hasta que se alcanzan unidades de código manejables.
4. **Desarrollo de módulos:** a partir de la descomposición, se implementan los módulos, primero los de nivel más alto, y luego se avanza hacia los niveles inferiores.
5. **Integración:** una vez que todos los módulos están desarrollados, se integran para formar el sistema completo.
6. **Pruebas y validación:** se realizan pruebas para asegurar que el sistema funcione correctamente y cumpla con los requisitos especificados.

Ejemplo: controlador de semáforo



Ventajas

- Dividir los problemas en partes ayuda a **identificar qué se debe hacer**.
- Cada paso de refinamiento **reduce la complejidad** y facilita la solución.
- Partes de la solución pueden ser **reutilizables**.
- Permite que **varias personas trabajen en diferentes partes** del problema.
- Es útil cuando se tiene una **comprensión general del sistema**, pero los detalles de implementación aún no están definidos.
- Es adecuado para proyectos con **estructuras bien definidas** y que pueden **descomponerse de manera lógica** en módulos.

Ejemplo: controlador de semáforo - código I

```
45 int main(void)
46 {
47     /* MCU Configuration-----*/
48
49     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
50     HAL_Init();
51
52     /* Configure the system clock */
53     SystemClock_Config();
54
55     /* Initialize all configured peripherals */
56     MX_GPIO_Init();
57     MX_USART2_UART_Init();
58
59     SEM_FSM_Init();
60
61
62     /* Infinite loop */
63     while (1)
64     {
65
66         SEM_FSM_Update();
67
68     }
69
70 }
```

Ejemplo: controlador de semáforo - código II

```
.c main.c  .h SEM.h ×  .c SEM.c
1  /*
2   * SEM.h
3   *
4   * Created on: 2 ago. 2023
5   * Author: Patricio
6   */
7
8  #ifndef API_INC_SEM_H
9  #define API_INC_SEM_H
10
11 void SEM_FSM_Init( void );
12
13 void SEM_FSM_Update( void );
14
15 #endif /* API_INC_SEM_H */
16
```

```
.c main.c  .h SEM.h  .c SEM.c ×
1  /*
2   * SEM.c
3   *
4   * Created on: 2 ago. 2023
5   * Author: patricio
6   */
7
8  #include "SEM.h"
9
10 void SEM_FSM_Init( void ){
11 };
12
13 void SEM_FSM_Update( void ){
14
15 };
16
```

Ejemplo: controlador de semáforo - código III

```
8 #ifndef API_INC_SEM_H_
9 #define API_INC_SEM_H_
10
11 typedef enum {
12     RED,
13     RED_YELLOW,
14     GREEN_YELLOW,
15     OFFLINE} SEMstate_t;
16
17 #define INITIAL_STATE OFFLINE
18
19 void SEM_FSM_Init( void );
20
21 void SEM_FSM_Update( void );
22
23 #endif /* API_INC_SEM_H_ */
24
```

```
8 #include "SEM.h"
9
10 static SEMstate_t state;
11
12 void SEM_FSM_Init( void ){
13     state = INITIAL_STATE;
14 };
15
16 void SEM_FSM_Update( void ){
17
18     switch(state){
19     case RED:
20         break;
21
22     case RED_YELLOW:
23         break;
24
25     case GREEN:
26         break;
27
28     case YELLOW:
29         break;
30
31     case OFFLINE:
32         break;
33
34     default:
35         break;
36
37     }
38 };

```